

Assignment 4 evidence

YihaoYin (yihaoyin)

generated 2026-09-24 22:06 ????? on Windows-11-10.0.26200-SP0, polars 1.44.2

project System & Tool Chains

evidence script sha256 76151d2e3fb565930aed34c3b78d3a4f8debede7fcdaae3d053cfd814d921ab8

Score 4.50 / 5 automatic, out of 6 for the assignment.

The last 1 point is REPORT.md, which your TA reads. This script does not score it.

Summary

group	score	checks	decided by
The table	1.00 / 1.00	4/4 passed	script
Baselines	1.00 / 1.00	4/4 passed	script
Models	1.25 / 1.25	5/5 passed	script
Honest evaluation	1.25 / 1.25	5/5 passed	script
Report numbers	0.00 / 0.50	0/2 passed	script
REPORT.md	? / 1.00	0/1 passed	your TA

automatic total	4.50 / 5.00	of 6 for the assignment	

What this script found

```
data      Week4\HW4\data\tep_fault_free_training.parquet
predictions Week4\HW4\results\predictions.parquet
forecast   Week4\HW4\results\forecast.csv
validation Week4\HW4\results\validation.csv
leaky      Week4\HW4\results\leaky.csv
report     Week2\HW2\REPORT.md
code       28 files
```

The table 1.00 / 1.00

```
[PASS] predictions.parquet with the eight columns
[PASS] every test run and origin, for every horizon, and nothing else
[PASS] y_true is xmeas_18 at t + h
      100.00% of rows match
[PASS] shifts kept inside each run in the code
```

Baselines 1.00 / 1.00

```
[PASS] forecast.csv with the five horizons and the six columns
[PASS] persistence RMSE matches the recomputation
      within 0.5 % at every horizon
[PASS] mean RMSE matches the recomputation
      mean over all samples of runs 1 to 300
[PASS] skill = 1 - direct / better baseline
```

Models 1.25 / 1.25

```
[PASS] direct RMSE matches your predictions
[PASS] recursive RMSE matches your predictions
[PASS] direct beats the better baseline at four or more horizons
      5 of 5
[PASS] recursive differs from direct beyond h = 1
[PASS] a scaler inside a pipeline in the code
```

Honest evaluation 1.25 / 1.25

```
[PASS] validation.csv with three or more settings
[PASS] validation on runs 301 to 400 in the code
[PASS] leaky.csv with both splits for runs 1 to 10
[PASS] TimeSeriesSplit with a gap of at least 10 in the code
      gap arguments found: ['H']
[PASS] shuffled KFold in the code
```

Report numbers 0.00 / 0.50

- [FAIL] RMSE values in REPORT.md come from forecast.csv
0 of 20 quoted; four needed
- [FAIL] REPORT.md quotes both leaky.csv means

REPORT.md 1.00 point, read by your TA

- [SKIP] six sections and the AI-use line
read by your TA

Each line is decided from the output below, not asserted.

Transcript

data, and the script's own baselines

```
250,000 rows, 500 runs, samples per run [500]
training mean of xmeas_18 over runs 1 to 300: 65.80559
h= 1: persistence 0.0501, mean 0.4231 on 46,000 test origins
h= 5: persistence 0.1400, mean 0.4244 on 45,600 test origins
h=10: persistence 0.2218, mean 0.4260 on 45,100 test origins
h=20: persistence 0.3920, mean 0.4289 on 44,100 test origins
h=40: persistence 0.6353, mean 0.4345 on 42,100 test origins
```

predictions

```
222,900 rows, columns ['run', 't', 'h', 'y_true', 'persistence', 'mean', 'direct', 'recursive']
runs: 401 to 500, 100 distinct
h= 1: 46,000 rows (expected 46,000), 0 duplicated origins, 0 origins outside 40..499
h= 5: 45,600 rows (expected 45,600), 0 duplicated origins, 0 origins outside 40..495
h=10: 45,100 rows (expected 45,100), 0 duplicated origins, 0 origins outside 40..490
h=20: 44,100 rows (expected 44,100), 0 duplicated origins, 0 origins outside 40..480
h=40: 42,100 rows (expected 42,100), 0 duplicated origins, 0 origins outside 40..460
y_true equal to xmeas_18 at t + h: 100.00% of rows
```

forecast.csv against the script's recomputation

```
columns ['h', 'minutes', 'rows', 'persistence_rmse', 'mean_rmse', 'direct_rmse', 'recursive_rmse',
'skill', 'recursive_skill']
```

h	persistence yours / script	mean yours / script	direct yours / from preds	recursive yours / from preds
1	0.0501 / 0.0501	0.4231 / 0.4231	0.0367 / 0.0367	0.0393 / 0.0393
5	0.1400 / 0.1400	0.4244 / 0.4244	0.0869 / 0.0869	0.1070 / 0.1070
10	0.2218 / 0.2218	0.4260 / 0.4260	0.1271 / 0.1271	0.1486 / 0.1486
20	0.3920 / 0.3920	0.4289 / 0.4289	0.2029 / 0.2029	0.2482 / 0.2482
40	0.6353 / 0.6353	0.4345 / 0.4345	0.3328 / 0.3328	0.3846 / 0.3846

validation.csv

```
24 settings, columns ['h', 'n_lags', 'alpha', 'features', 'train_rows', 'train_rmse', 'val_rmse',
'val_persistence_rmse', 'chosen']
```

```
shape: (12, 9)
????????????????????????????????????????????????????????????????????????????????????
? h ? n_lags ? alpha ? features ? ? ? train_rmse ? val_rmse ? val_persistence_rmse ? chosen ?
? --- ? --- ? --- ? ? --- ? --- ? --- ? --- ?
? i64 ? i64 ? f64 ? i64 ? ? f64 ? f64 ? f64 ? bool ?
????????????????????????????????????????????????????????????????????????????????????
? 10 ? 1 ? 0.01 ? 12 ? ? ? 0.154756 ? 0.152825 ? 0.227914 ? false ?
? 10 ? 1 ? 1.0 ? 12 ? ? ? 0.154756 ? 0.152825 ? 0.227914 ? false ?
? 10 ? 1 ? 100.0 ? 12 ? ? ? 0.154768 ? 0.152833 ? 0.227914 ? false ?
? 10 ? 1 ? 10000.0 ? 12 ? ? ? 0.16248 ? 0.160314 ? 0.227914 ? false ?
? 10 ? 2 ? 0.01 ? 13 ? ? ? 0.149295 ? 0.14773 ? 0.227914 ? false ?
? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
? 10 ? 2 ? 10000.0 ? 13 ? ? ? 0.164293 ? 0.162113 ? 0.227914 ? false ?
```

[illegible]

leaky.csv

```
20 rows, columns ['run', 'split', 'h', 'model_rmse', 'persistence_rmse']
runs with a shuffled row: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
runs with a time-ordered row: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```

shape: (2, 3)
????????????????????????????????????????
? kind      ? model_rmse ? persistence_rmse ?
? ---      ? ---      ? ---      ?
? str       ? f64      ? f64      ?
????????????????????????????????????????
? shuffled ? 0.096044   ? 0.255007
? time     ? 0.344777   ? 0.251701
????????????????????????????????????????

```

read the code

28 code files: Week1\sensorlab\01-evidence.py, Week1\sensorlab\src\sensorlab__init__.py,
Week1\sensorlab\src\sensorlab\dataprocess.py, Week1\sensorlab\src\sensorlab\train.py,
Week2\HW2\src\comparison.py, Week2\HW2\src\duckdb_export.py, Week2\HW2\src\load.py,
Week2\HW2\src\query.py, Week3\HW3\src\clean.py, Week3\HW3\src\contract.py, Week3\HW3\src\decode.py,
Week3\HW3\src\disturbance.py

```
shifts kept inside a run (.over / group_by / per-run loop): yes
scaler inside a pipeline: yes
validation runs 301 to 400 referenced: yes
TimeSeriesSplit: yes; gap arguments: ['H']
KFold with shuffle=True: yes
```

REPORT.md

```
1441 words, 17 headings: ['Report', 'Part 1 Schema', 'Results', 'Discussion', 'Part 2 Load and Cleaning',
    'Results', 'Discussion', 'Part 3 Indexing', 'Method', 'Results']
RMSE values from forecast.csv quoted in the report: 0 of 20
leaky.csv means quoted: shuffled no, time no
AI-use line: yes
```

Code: Week1\sensorlab\01-evidence.py

```
#!/usr/bin/env python3
"""Build the Assignment 1 evidence report.
```

Run this in the root of your project, the directory holding 'pyproject.toml':

```
python3 a01-evidence.py --andrew-id yourid --name "Your Name"
```

It writes 'evidence.pdf'. Upload that to Canvas. That PDF is your submission, so read it before you send it.

WHAT IT DOES. It runs the commands in the assignment's Definition of done and records what they actually printed, then embeds the files a grader needs to read: your 'pyproject.toml', your package, the code cells of your notebook, and your README. Nothing is invented. If a command fails, the failure goes in the report, which is better for you than a report that quietly omits it: a report showing one broken command and five working ones is worth more marks than no report at all.

It deletes `.venv` and rebuilds it with `uv sync --locked`, because that rebuild is what the first 30 points are for. That is the only destructive thing it does, and it is the same thing the assignment asks you to do by hand at least once.

NAMES. The assignment prescribes names ('sensorlab', 'src/sensorlab/train.py', 'notebooks/explore.ipynb', 'README.md', 'sqlite:///mlflow.db') and this script looks for those first. It also goes looking when it does not find them, so a project laid out sensibly under different names still produces a report rather than a page of failures. What it found is printed at the top of the report, and every guess can be overridden with --module, --notebook or --tracking-uri. If the header says it used something you did not intend, pass the flag and run it again.

The PDF is written here, with no browser and no LaTeX and nothing to install, so every submission arrives in the same shape and there is no print dialog to get wrong.

Standard library only, so it runs on the system Python without being installed into your project. It shells out to 'uv', and to nothing else.

```
"""
```

```
from __future__ import annotations
```

```
import argparse
import hashlib
import html
import json
import platform
import re
import shutil
import subprocess
import sys
import time
from datetime import datetime, timezone
from pathlib import Path
```

```
TIMEOUT = 900 # seconds, per command; a cold uv cache can take a few minutes
SKIP_DIRS = {"venv", ".git", "node_modules", "__pycache__", ".ipynb_checkpoints",
             "build", "dist", ".pytest_cache", "mlruns", "site-packages"}
DATA_SUFFIXES = (".csv", ".tsv", ".xlsx", ".xls", ".zip", ".parquet", ".data",
                 ".gz", ".h5", ".nc")
# The project's own machinery, which this assignment tells students to create and
# which the data check must never mistake for a raw file. An MLflow store is a
# SQLite database, so ".sqlite" cannot live in DATA_SUFFIXES above: a student who
# named their store mlflow.sqlite instead of mlflow.db was told their MLflow store
# was stray raw data, and lost the data check and the cross-check together.
# uv.lock is here for the size rule rather than the suffix rule: with pandas,
# scikit-learn, matplotlib and mlflow resolved it runs to hundreds of kilobytes.
OURS_BY_SUFFIX = (".db", ".sqlite", ".sqlite3", ".lock")
BIG_FILE = 1_000_000 # a tracked file this large is data by any other name
OURS = ("a01-evidence.py", "evidence.html", "evidence.pdf")
# Finder and Explorer leave these behind in any directory a student opens.
# They say nothing about the submission, and failing the clean-tree check
# over one would be a mark deducted for using a Mac.
OS_JUNK = {".DS_Store", "Thumbs.db", "desktop.ini", ".Spotlight-V100", ".Trashes"}
```

```
class Step:
```

```
    """One recorded command: what was run, what came back, how long it took."""
```

```
    def __init__(self, label, command, stdout, code, seconds):
        self.label = label
        self.command = command
        self.stdout = stdout
        self.code = code
        self.seconds = seconds
```

```
    @property
```

```
    def ok(self):
        return self.code == 0
```

```

def run(label, command, cwd, steps):
    """Run one command, record it, and return the Step."""
    print(f" {label} ...", flush=True)
    start = time.monotonic()
    try:
        proc = subprocess.run(
            command, cwd=cwd, capture_output=True, text=True, timeout=TIMEOUT
        )
        out = (proc.stdout or "") + (proc.stderr or "")
        # uv's VIRTUAL_ENV warning fires for anyone who happens to have another
        # environment active. It says nothing about the submission.
        out = "\n".join(
            line for line in out.splitlines()
            if "VIRTUAL_ENV" not in line and "use '--active'" not in line
        )
        code = proc.returncode
    except FileNotFoundError:
        out, code = f"{command[0]}: not found on PATH", 127
    except subprocess.TimeoutExpired:
        out, code = f"timed out after {TIMEOUT}s", 124
    # A python -c one-liner is unreadable as a command line, so show its head.
    shown = " ".join(command)
    if "-c" in command:
        shown = " ".join(command[:command.index("-c") + 1]) + ' "..."'
    step = Step(label, shown, out.strip(), code, time.monotonic() - start)
    steps.append(step)
    return step

# ----- discovery

def _usable(path):
    return not any(part in SKIP_DIRS for part in Path(path).parts)

def find_module(root, given=None):
    """The module to run with 'python -m', and the package directory it is in.

    src/<pkg>/ first, because that is what the assignment asks for, then a
    package beside pyproject.toml so a flat layout still reports. Within the
    package, train.py wins, then any module that both parses arguments and has a
    main, since that is what an entry point looks like from the outside.
    """
    packages = [p.parent for p in sorted(root.glob("src/*/__init__.py")) if _usable(p)]
    packages += [p.parent for p in sorted(root.glob("*/__init__.py")) if _usable(p)]
    package = packages[0] if packages else None
    if given or package is None:
        return given, package

    modules = [m for m in sorted(package.glob("*.py")) if m.stem != "__init__"]
    entry = next((m for m in modules if m.stem == "train"), None)
    if entry is None:
        for candidate in modules:
            text = candidate.read_text(errors="replace")
            if "argparse" in text and "def main" in text:
                entry = candidate
                break
    if entry is None:
        return package.name, package
    return f"{package.name}.{entry.stem}", package

def find_notebook(root, given=None):
    """The exploratory notebook: the prescribed path first, then anything plausible."""

```

```

if given and (root / given).exists():
    return root / given
prescribed = root / "notebooks" / "explore.ipynb"
if prescribed.exists():
    return prescribed
found = [p for p in sorted(root.rglob("*.ipynb")) if _usable(p)]
if not found:
    return None
# Under notebooks/ first, then the largest, which is the exploratory one
# rather than somebody's scratch file.
found.sort(key=lambda p: (0 if p.parent.name == "notebooks" else 1, -p.stat().st_size))
return found[0]

def find_readme(root):
    """README.md, readme.md, README.rst, README.txt, or plain README."""
    for candidate in sorted(root.glob(".*")):
        if candidate.is_file() and candidate.stem.lower() == "readme":
            return candidate
    return None

def find_tracking_uri(root, given=None):
    """Where MLflow put the runs.

    The assignment prescribes sqlite:///mlflow.db. A student who used the file
    store instead still has runs worth showing, and reporting "no runs found"
    for a project that logged them correctly would cost ten points over a path
    this script guessed rather than anything they did.
    """
    if given:
        return given

    # Newest wins, not the prescribed name. Called after the runs, the store
    # they just wrote to is the one with the newest mtime, so a leftover
    # mlflow.db from an earlier attempt cannot send the search to an empty
    # database while the real runs sit in the file beside it.
    # Any SQLite suffix, not just .db. Naming the store mlflow.sqlite is an
    # ordinary convention, and looking only for .db meant the runs were never
    # found and the cross-check failed over the spelling of a filename.
    candidates = [(p.stat().st_mtime, f"sqlite:///[{p.name}]")
                  for pattern in ("*.db", "*.sqlite", "*.sqlite3")
                  for p in root.glob(pattern) if _usable(p) and p.is_file()]
    mlruns = root / "mlruns"
    if mlruns.is_dir():
        candidates.append((mlruns.stat().st_mtime, "file:./mlruns"))
    if candidates:
        return max(candidates)[1]
    return "sqlite:///mlflow.db"

# ----- reading

def sha256(path):
    path = Path(path) if path else None
    return hashlib.sha256(path.read_bytes()).hexdigest()[:16] if path and path.exists() else "absent"

def self_hash():
    """This file's own sha256, printed in the report.

    HONESTLY, WHAT THIS BUYS. Not much on its own. Anyone who edits the script
    can also edit this function, and the hash is computed on the machine doing
    the editing. What it catches is the cheap case: a student who changes a
    command or a check and does not think about the header line. The published
    checksum sits beside the script on the course site, so a grader comparing

```

one line spots that in a second.

The expensive-to-forge part of this report is not the hash. It is the cross-checks, which tie the transcript to the MLflow table and to the notebook, and which all have to agree with each other.

```
"""
return hashlib.sha256(Path(__file__).resolve().read_bytes()).hexdigest()
```

```
def read(path, limit=20000):
    """File contents, truncated, or a note saying it is missing."""
    if path is None or not Path(path).exists():
        return f"[{path} is not in this project]"
    text = Path(path).read_text(errors="replace")
    return text if len(text) <= limit else text[:limit] + "\n[truncated]"

def trim(text, head=14, tail=8):
    """Long output, shortened for print.

    'uv sync' lists every package it installed, which is 169 lines for this
    project and pages of a PDF nobody reads. The count that matters and the
    lines either side of the elision are kept.
    """
    lines = text.splitlines()
    if len(lines) <= head + tail + 3:
        return text
    elided = len(lines) - head - tail
    return "\n".join(lines[:head] + [f"    [... {elided} lines elided ...]" ] + lines[-tail:])

def numbers(text):
    """Every decimal number in some output, rounded, as a set.

    Comparing numbers rather than whole strings is what lets the determinism
    check survive a student who prints a run id or a timestamp beside the
    metric. Comparing whole strings called those submissions non-deterministic,
    which they were not.

    One decimal place counts, and scientific notation counts. Requiring two
    digits meant "R2 = 0.8" and "R2 = 1.0" produced no numbers at all, which
    failed both determinism checks and the cross-check together: half the score,
    for printing a metric with one decimal place in a project that was correct.
    Rounding to four places is what keeps 0.8 and 0.80 the same number.
    """
    pattern = r"-?\d+\.\d+(?:[eE][-+]?[d+]?|-?\d+[eE][-+]?[d+]"
    return {f"{float(t):.4f}" for t in re.findall(pattern, text)}

def notebook_code(path):
    """The notebook's code cells, with their execution counts.

    The execution counts are the interesting part. A notebook run top to bottom
    from a fresh kernel numbers its code cells 1, 2, 3 and so on. Any other
    pattern means the cells were run out of order, which is the hidden-state
    problem from Lecture 1.
    """
    if path is None or not Path(path).exists():
        return "[no notebook found in this project]", None
    try:
        cells = json.loads(Path(path).read_text(errors="replace"))["cells"]
    except (ValueError, KeyError, TypeError):
        return f"[{path} is not readable as a notebook]", None

    counts, blocks = [], []
    for cell in cells:
        if cell.get("cell_type") != "code":
```

```

        continue
    count = cell.get("execution_count")
    counts.append(count)
    blocks.append(f"In [{count if count is not None else ' '}]:\n"
                  + "".join(cell.get("source", [])))
if not counts:
    return "[the notebook has no code cells]", None
return "\n\n".join(blocks), counts == list(range(1, len(counts) + 1))

def package_source(root, package):
    """Every .py in the student's package, plus any scripts/ they wrote."""
    if package is None:
        return "[no package directory found]"
    files = [p for p in sorted((root / package).rglob("*.py")) if _usable(p)]
    files += [p for p in sorted(root.glob("scripts/*.py")) if _usable(p)]
    if not files:
        return f"[no Python files under {package}]"
    return "\n\n".join(f"--- {p.relative_to(root)}\n{read(p, limit=12000)}" for p in files)

# ----- collection

def collect(root, found):
    """Run every command the report is built from."""
    steps = []
    run("tool versions", ["uv", "--version"], root, steps)

    venv = root / ".venv"
    if venv.exists():
        shutil.rmtree(venv)
    # --locked, not a bare sync. A bare 'uv sync' silently WRITES a lockfile if
    # one is missing, so a project with no committed lockfile would rebuild
    # happily and the check that is supposed to fail would pass.
    rebuild = run("rebuild the environment from the lockfile",
                  ["uv", "sync", "--locked"], root, steps)
    if not rebuild.ok:
        # --locked fails when the lockfile is missing OR merely out of date with
        # pyproject.toml, which is one mistake. Without a fallback it also takes
        # the runs, both determinism checks and the cross-check down with it, so
        # one mistake would cost 70 points instead of 30. Sync anyway, and let
        # the lockfile check stay failed on its own.
        run("lockfile rejected, so rebuilding without --locked to continue",
            ["uv", "sync"], root, steps)

    run("resolved versions",
        ["uv", "run", "python", "-c",
         "import sys\n"
         "print('python', sys.version.split()[0])\n"
         "for name in ('numpy', 'pandas', 'sklearn', 'mlflow'):\n"
         "    try:\n"
         "        print(name, __import__(name).__version__)\n"
         "    except Exception as exc:\n"
         "        print(name, 'not importable:', exc)"],
        root, steps)

    module = found["module"]
    blank = Step("run", "", "no entry point found, so the runs were skipped", 1, 0.0)
    first = again = other = blank
    if module:
        first = run("run, seed 0", ["uv", "run", "python", "-m", module, "--seed", "0"], root, steps)
        again = run("run, seed 0 again", ["uv", "run", "python", "-m", module, "--seed", "0"], root, steps)
        other = run("run, seed 1", ["uv", "run", "python", "-m", module, "--seed", "1"], root, steps)
    else:
        steps.append(blank)

```

```

# Detected here rather than before the runs: a file store does not exist
# until something has been logged to it, so detecting up front sent a
# perfectly good project to the wrong place and reported "no runs found".
if not found.get("tracking_given"):
    found["tracking_uri"] = find_tracking_uri(root)

mlflow_step = run(
    "MLflow runs",
    ["uv", "run", "python", "-c",
     "import mlflow, pandas as pd\n"
     f"mlflow.set_tracking_uri({found['tracking_uri']!r})\n"
     "pd.set_option('display.width', 200)\n"
     "df = mlflow.search_runs(search_all_experiments=True)\n"
     "if len(df):\n"
     "    df = df.sort_values('start_time', ascending=False).head(10)\n"
     "    cols = [c for c in ['run_id', 'params.seed', 'metrics.r2', 'start_time'] if c in
df.columns]\n"
     "    cols += [c for c in df.columns if c.startswith('metrics.') and c not in cols][:2]\n"
     "    print(df[cols].to_string(index=False))\n"
     "else:\n"
     "    print('no runs found')"],
    root, steps)

nb_source, nb_in_order = notebook_code(found["notebook_path"])

# Hygiene from the filesystem rather than from git, because Assignment 1 no
# longer asks anyone to run git. The lesson is the same one L2 teaches, that
# data has one home and code has another, and it is checkable without a
# repository: the raw file belongs under data/, which the download creates,
# and nowhere else in the project.
stray = []
for path in sorted(root.rglob("*")):
    if not path.is_file() or not _usable(path) or path.name in OS_JUNK:
        continue
    relative = path.relative_to(root)
    if relative.parts[0] in ("data", *OURS) or relative.name in OURS:
        continue
    if path.suffix.lower() in OURS_BY_SUFFIX:
        # A lockfile or an MLflow store. Both are supposed to be here, and
        # both are large enough to trip the size rule below on an ordinary
        # project: this reference one has an 872 kB store after three runs,
        # growing about 5 kB a run, and a 786 kB lockfile. Deducting for
        # either is a mark taken for following the instructions.
        continue
    if path.suffix.lower() in DATA_SUFFIXES:
        stray.append(str(relative))
    elif path.stat().st_size > BIG_FILE:
        # A renamed data file has no telling suffix, so size is the tell.
        stray.append(f"{relative} ({path.stat().st_size // 1000} kB)")

# Do the numbers the seed 0 run printed turn up again in the MLflow table?
# Two independent records of one run, so a forged number has to be forged in
# both places, consistently, in a format the student did not choose. That is
# a better control than any hash of this file.
printed = numbers(first.stdout)
logged = numbers(mlflow_step.stdout)

checks = [
    ("The environment rebuilds from the lockfile",
     rebuild.ok and (root / "uv.lock").is_file() and (root / ".python-version").is_file()),
    ("The same seed prints the same number twice",
     first.ok and again.ok and bool(printed) and printed == numbers(again.stdout)),
    ("A different seed prints a different number",
     other.ok and bool(printed) and numbers(other.stdout) != printed),
    ("The raw data sits under data/ and nowhere else in the project", not stray),
    ("The notebook's cells ran in order from a fresh kernel", bool(nb_in_order)),

```

```

        ("The metric printed by the seed 0 run also appears in the MLflow table",
         bool(printed & logged)),
    ]

    notes = []
    if stray:
        notes.append("data outside data/: " + ", ".join(stray[:5]))
    return steps, checks, nb_source, notes

# ----- colour and highlight

BLACK = (0, 0, 0)
GREEN = (0.00, 0.45, 0.15)
RED = (0.70, 0.06, 0.06)
BLUE = (0.10, 0.25, 0.60)
GREY = (0.40, 0.40, 0.40)
KEYWORD = (0.45, 0.15, 0.55)
STRING = (0.62, 0.20, 0.12)
NUMBER = (0.05, 0.40, 0.50)
COMMENT = (0.35, 0.48, 0.38)

KEYWORDS = {
    "False", "None", "True", "and", "as", "assert", "async", "await", "break",
    "class", "continue", "def", "del", "elif", "else", "except", "finally",
    "for", "from", "global", "if", "import", "in", "is", "lambda", "nonlocal",
    "not", "or", "pass", "raise", "return", "try", "while", "with", "yield",
}
TOKENS = re.compile(
    r"(?P<comment>#[^\n]*)"
    r"|(?P<string>'\"|\"\\\"|'(?:\\"|\"[^\"]\\")*\"|'(?:\\"|\"[^\"]\\")*\"'"
    r"|(?P<number>\b\d+\.\d*\b)"
    r"|(?P<word>[A-Za-z_]\w*)"
)

def plain(text, bold=False, colour=BLACK):
    """One line, one colour. Lines are lists of (text, bold, colour) runs."""
    return [(text, bold, colour)]

def highlight(source, indent=" "):
    """Python source as coloured runs, one list of runs per line.

    A hand-rolled highlighter rather than a dependency, and deliberately a small
    one: keywords, strings, numbers and comments. It tracks triple-quoted
    strings across lines, because docstrings are the one multi-line construct
    that every student's module will contain.
    """
    out, in_triple = [], None
    for raw in source.splitlines():
        runs, position = [(indent, False, BLACK)], 0
        if in_triple:
            end = raw.find(in_triple)
            if end == -1:
                out.append([(indent, False, BLACK), (raw, False, STRING)])
                continue
            runs.append((raw[:end + 3], False, STRING))
            position = end + 3
            in_triple = None
        for match in TOKENS.finditer(raw, position):
            # finditer was started before a triple-quoted run was consumed by
            # hand, so it still yields matches from inside it. Skipping those is
            # what stops a one-line docstring being printed twice.
            if match.start() < position:
                continue
            if match.start() > position:

```

```

        runs.append((raw[position:match.start()], False, BLACK))
    text = match.group()
    kind = match.lastgroup
    if kind == "comment":
        runs.append((text, False, COMMENT))
    elif kind == "string":
        if text in ('"', "'"):
            rest = raw[match.start():]
            closing = rest.find(text, 3)
            if closing == -1:
                runs.append((rest, False, STRING))
                in_triple = text
                position = len(raw)
                break
            text = rest[:closing + 3]
            runs.append((text, False, STRING))
            position = match.start() + len(text)
            continue
        runs.append((text, False, STRING))
    elif kind == "number":
        runs.append((text, False, NUMBER))
    elif text in KEYWORDS:
        runs.append((text, True, KEYWORD))
    else:
        runs.append((text, False, BLACK))
    position = match.end()
    if position < len(raw):
        runs.append((raw[position:], False, BLACK))
    out.append(runs)
    return out

```

----- PDF output

```

def escape_pdf(text):
    out = text.replace("\\", r"\\").replace("(", r"\(").replace(")", r"\)")
    return out.encode("latin-1", "replace").decode("latin-1")

```

```

def wrap(lines, max_chars, indent="    "):
    """Wrap coloured lines to the page width, keeping each run's colour.

    Courier is monospace, so a character count is a width, and a line can be cut
    anywhere without measuring anything.
    """
    out = []
    for runs in lines:
        current, used = [], 0
        for text, bold, colour in runs:
            text = text.replace("\t", " ")
            while True:
                room = max_chars - used
                if len(text) <= room:
                    if text:
                        current.append((text, bold, colour))
                        used += len(text)
                    break
                cut = text.rfind(" ", 0, room)
                cut = cut if cut > room // 2 else room
                current.append((text[:cut], bold, colour))
                out.append(current)
                current, used = [(indent, False, BLACK)], len(indent)
                text = text[cut:].lstrip()
            out.append(current)
    return out

```

```
def write_pdf(path, lines, title):
    """Write a paginated, coloured, monospace PDF.

    No dependency, no browser, no LaTeX, so every submission arrives in the same
    shape and there is no print dialog to get wrong. 'lines' is a list of lines,
    each a list of (text, bold, colour) runs. Successive Tj operators continue
    where the last o
    [truncated]
```

Code: Week1\sensorlab\src\sensorlab__init__.py

```
def main() -> None:
    print("Hello from sensorlab!")
```

Code: Week1\sensorlab\src\sensorlab\dataprocess.py

```
import numpy as np
import pandas as pd
import io
import urllib.request
import zipfile
from pathlib import Path

URL = 'https://archive.ics.uci.edu/static/public/360/air+quality.zip'

def load(path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    if not path.exists():
        print(f'fetching {URL}')
        with urllib.request.urlopen(URL) as response:
            payload = response.read()
        with zipfile.ZipFile(io.BytesIO(payload)) as archive:
            path.write_bytes(archive.read('AirQualityUCI.csv'))

    return pd.read_csv(path, sep=';', decimal=',')

def clean(df):
    df = (df.dropna(axis=1, how='all').dropna(how='all'))

    df['ts'] = pd.to_datetime(
        df['Date'] + ' ' + df['Time'].str.replace('.', ':', regex=False),
        format='%d/%m/%Y %H:%M:%S',
    )
    df = df.replace(-200, np.nan)

    return df
```

Code: Week1\sensorlab\src\sensorlab\train.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from .dataprocess import clean, load
from pathlib import Path
import mlflow
import argparse

def train(data_path, seed):
    df = load(data_path)
    df = clean(df)

    d = df.dropna(subset=['CO(GT)', 'PT08.S1(CO)'])
```

```

X = d[['PT08.S1(CO)']]
y = d['CO(GT)']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=seed)

model = LinearRegression().fit(X_train, y_train)
score = r2_score(y_test, model.predict(X_test))

return model, score

# Argument Part is generated by ChatGPT
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--seed", type=int, default=0)
    parser.add_argument(
        "--data-path",
        default="data/AirQualityUCI.csv",
    )
    args = parser.parse_args()

    model, score = train(args.data_path, args.seed)

    mlflow.set_tracking_uri("sqlite:///mlflow.db")
    mlflow.set_experiment("sensorlab")

    with mlflow.start_run(run_name=f"seed-{args.seed}"):
        mlflow.log_params({"seed": args.seed, "data": Path(args.data_path).name})
        mlflow.log_metric("r2", score)

    print(f"r2={score}")
    print(f"slope {model.coef_[0]:.5f}, "f"intercept {model.intercept_:.5f}")

if __name__ == '__main__':
    main()

```

Code: Week2\HW2\src\comparison.py

```

# uv run python src/comparison.py

import time

import duckdb
import pandas as pd

from load import bench, connect

GLOB = "parquet/readings/**/*.parquet"
NEEDED = ["sensor_id", "variable", "value"] # the only three columns this question names
LOC = {"pandas": 1, "SQLite": 2, "DuckDB": 2} # counted by hand from the three answers below

# ----- The same question, three engines -----

def pandas_answer(df):
    return df[df["variable"] == "temperature"].groupby("sensor_id")["value"].mean()

def sqlite_answer(conn):
    return conn.execute("SELECT sensor_id, avg(value) FROM readings "
                        "WHERE variable = 'temperature' GROUP BY sensor_id").fetchall()

def duckdb_answer(con):
    return con.execute(f"SELECT sensor_id, avg(value) FROM '{GLOB}' "
                      f"WHERE variable = 'temperature' GROUP BY sensor_id").fetchall()

```

```
# ----- Time them -----
```

```
def main():
    t = time.perf_counter()
    df = pd.read_parquet("parquet/readings", engine="pyarrow", columns=NEEDED)
    read_s = time.perf_counter() - t

    conn = connect()
    con = duckdb.connect()

    print("Query: average temperature per mote. Timed warm, best of 5, the query only.\n")
    print("| engine | query (ms) | lines of code |")
    print("|---|---|---|")
    for label, fn, arg in (("pandas", pandas_answer, df),
                           ("SQLite", sqlite_answer, conn),
                           ("DuckDB", duckdb_answer, con)):
        print(f"| {label} | {bench(lambda: fn(arg)):.1f} | {LOC[label]} |")

    # the same three answers, sorted by sensor, so the table above compares one question and not three
    a_pandas = sorted((int(k), float(v)) for k, v in pandas_answer(df).items())
    a_sqlite = sorted((int(k), float(v)) for k, v in sqlite_answer(conn))
    a_duckdb = sorted((int(k), float(v)) for k, v in duckdb_answer(con))
    gap_sqlite = max(abs(x[1] - y[1]) for x, y in zip(a_pandas, a_sqlite))
    gap_duckdb = max(abs(x[1] - y[1]) for x, y in zip(a_pandas, a_duckdb))

    print(f"\npandas first read {len(df):,} rows into RAM, which took {read_s:.1f} s.")
    print(f"{len(a_pandas)} motes, largest gap: pandas vs SQLite {gap_sqlite:.3g}, "
          f"pandas vs DuckDB {gap_duckdb:.3g}.")
    conn.close()

if __name__ == "__main__":
    main()
```

Code: Week2\HW2\src\duckdb_export.py

```
# uv run python src/duckdb_export.py

import shutil
import time
from pathlib import Path

import duckdb

DB = "lab.db"
OUT = Path("parquet/readings")
PARQUET = "read_parquet('parquet/readings/**/*.parquet', hive_partitioning = true)"
# hive_partitioning restores 'date', which exists only as a directory name

# query (a): DuckDB has a real TIMESTAMP type, so the text is cast rather than sliced
A = """
SELECT sensor_id,
       strftime(ts::TIMESTAMP, '%Y-%m-%d %H:00:00') AS hour,
       count(*) AS n,
       round(avg(value), 2) AS avg_temp
FROM {src}
WHERE variable = 'temperature'
GROUP BY ALL
"""

# query (c): the RANGE frame is ordered by ts_unix here too, never by the ts text
C = """
SELECT sensor_id, ts, value AS voltage,
       round(avg(value) OVER (PARTITION BY sensor_id ORDER BY ts_unix
                             RANGE BETWEEN 3600 PRECEDING AND CURRENT ROW), 4)
```

```

        AS voltage_1h_avg,
count(*)      OVER (PARTITION BY sensor_id ORDER BY ts_unix
                    RANGE BETWEEN 3600 PRECEDING AND CURRENT ROW)
                    AS n_in_window

FROM {src}
WHERE variable = 'voltage'
"""

# ----- Export to date-partitioned Parquet -----

def export(con):
    if OUT.exists():
        shutil.rmtree(OUT) # a partial tree left by an earlier run would be re-read
    OUT.parent.mkdir(parents=True, exist_ok=True)
    t = time.perf_counter()
    # date is the first 10 characters of ts, and becomes the partition directory name
    con.execute(f"""
        COPY (SELECT *, CAST(substr(ts, 1, 10) AS DATE) AS date FROM lab.readings)
        TO '{OUT.as_posix()}' (FORMAT parquet, PARTITION_BY (date))
    """)
    return time.perf_counter() - t

def timed(con, sql):
    t = time.perf_counter()
    rows = con.execute(sql).fetchall()
    return rows, (time.perf_counter() - t) * 1000

# ----- Reproduce (a) and (c) against the Parquet -----

def main():
    con = duckdb.connect()
    con.execute("INSTALL sqlite; LOAD sqlite;")
    con.execute(f"ATTACH '{DB}' AS lab (TYPE sqlite, READ_ONLY)") # no Python between the two engines

    export_s = export(con)
    parquet_mb = sum(f.stat().st_size for f in OUT.rglob("*.parquet")) / 1e6
    print(f"export      {export_s:,.1f} s into {len(list(OUT.glob('date=*')))} date partitions")
    print(f"on disk      lab.db {Path(DB).stat().st_size / 1e6:,.0f} MB, "
          f"parquet {parquet_mb:,.0f} MB")

    a_rows, a_ms = timed(con, A.format(src=PARQUET))
    print(f"(a)          {len(a_rows):,} hourly buckets, {a_ms:,.0f} ms")

    c_rows, c_ms = timed(con, C.format(src=PARQUET))
    window = [r[4] for r in c_rows] # ordering the frame by text would pin every window at 1
    print(f"(c)          {len(c_rows):,} rows, {c_ms:,.0f} ms, "
          f"n_in_window {min(window)}-{max(window)}")
    con.close()

if __name__ == "__main__":
    main()

```

Code: Week2\HW2\src\load.py

```

# uv run python src/load.py

import sqlite3
import time
from datetime import datetime, timezone
from pathlib import Path

DB = Path("lab.db")
DATA = Path("data/data.txt")

```

```

ROSTER_FILE = Path("data/mote_locs.txt")
SCHEMA = Path("sql/schema.sql")

CHANNELS = ("temperature", "humidity", "light", "voltage")
BATCH = 200_000
REPEATS = 5
LOW_BATTERY_V = 2.4      # counted and reported, never used to drop a row; REPORT.md says why

INSERT = ("INSERT INTO readings (sensor_id, ts, ts_unix, epoch, variable, value) "
          "VALUES (?, ?, ?, ?, ?, ?)")

# the natural key and the access path in one statement, built after the load
INDEX = "CREATE UNIQUE INDEX ux_readings ON readings (sensor_id, ts, variable)"
PROBE = "SELECT count(*) FROM readings WHERE sensor_id = ? AND ts >= ? AND ts < ?"
PROBE_ARGS = (22, "2004-03-09 00:00:00", "2004-03-10 00:00:00")

# ----- Helpers -----

def connect(path=DB):
    conn = sqlite3.connect(path, isolation_level=None)    # autocommit, so BEGIN/COMMIT are ours
    conn.execute("PRAGMA foreign_keys = ON")              # per connection, and off by default
    return conn

def bench(fn, n=REPEATS):
    fn()                                                    # warm up, then take the best of n
    best = float("inf")
    for _ in range(n):
        t = time.perf_counter()
        fn()
        best = min(best, (time.perf_counter() - t) * 1000)
    return best

def read_roster(path=ROSTER_FILE):
    roster = []
    for entry in path.read_text().splitlines():
        if entry.strip():
            mote, x, y = entry.split()                    # 'moteid x y', whitespace separated
            roster.append((int(mote), float(x), float(y)))
    return roster

def to_instant(day, clock):
    sec, _, frac = clock.partition(".")                    # four rows in data.txt land on a whole second
    ts = f"{day} {sec}.{(frac + '000000')[:6]}"            # pad to fixed width, so the text sorts by time
    return datetime.fromisoformat(ts).replace(tzinfo=timezone.utc).timestamp(), ts

# ----- Load dataset -----

def load(conn):
    n = dict(rows=0, ragged=0, no_mote=0, off_roster=0, unparseable=0, accepted=0, empty_cells=0)

    roster_rows = read_roster()
    roster = {r[0] for r in roster_rows}
    conn.execute("BEGIN")
    conn.executemany("INSERT INTO sensors (sensor_id, x_m, y_m) VALUES (?, ?, ?)", roster_rows)
    conn.execute("COMMIT")

    batch = []
    t0 = time.perf_counter()
    conn.execute("BEGIN")                                # one transaction for the whole file, not one per row
    with DATA.open("r", encoding="utf-8", errors="replace") as fh:
        for entry in fh:
            entry = entry.rstrip("\r\n")                  # not .strip(): an empty voltage leaves a trailing space

```

```

    if not entry:
        continue
    n["rows"] += 1
    f = entry.split(" ")
    if len(f) != 8:
        n["ragged"] += 1
        continue
    if not f[3].isdigit():
        n["no_mote"] += 1
        continue
    mote_id = int(f[3])
    if not (1 <= mote_id <= 54) or mote_id not in roster:
        n["off_roster"] += 1
        continue
    try:
        unix, ts = to_instant(f[0], f[1])
        epoch = int(f[2])
        cells = [None if c == "" else float(c) for c in f[4:8]]
    except ValueError:
        n["unparseable"] += 1
        continue
    n["accepted"] += 1
    n["empty_cells"] += cells.count(None) # an empty channel is absent, not zero
    batch += [(mote_id, ts, unix, epoch, name, v)
               for name, v in zip(CHANNELS, cells) if v is not None]
    if len(batch) >= BATCH:
        conn.executemany(INSERT, batch)
        batch.clear()
    conn.executemany(INSERT, batch)
    conn.execute("COMMIT")
    n["load_s"] = round(time.perf_counter() - t0, 1)
    return n

```

----- Cleaning report -----

```

def cleaning_report(conn, n):
    return {
        "rows_in_file": n["rows"],
        "ragged_rows": n["ragged"],
        "rejected_empty_moteid": n["no_mote"],
        "rejected_off_roster": n["off_roster"],
        "rejected_unparseable": n["unparseable"],
        "rows_accepted": n["accepted"],
        "absent_channel_cells": n["empty_cells"],
        "readings_loaded": conn.execute("SELECT count(*) FROM readings").fetchone()[0],
        "voltage_below_2.4V": conn.execute(
            "SELECT count(*) FROM readings WHERE variable = 'voltage' AND value < ?",
            (LOW_BATTERY_V,)).fetchone()[0],
        "foreign_key_violations": len(conn.execute("PRAGMA foreign_key_check").fetchall()),
        "load_seconds": n["load_s"],
    }

```

----- Index: one query, before and after -----

```

def probe(conn):
    plan = " ".join(r[3] for r in conn.execute("EXPLAIN QUERY PLAN " + PROBE, PROBE_ARGS))
    rows = conn.execute(PROBE, PROBE_ARGS).fetchone()[0]
    ms = bench(lambda: conn.execute(PROBE, PROBE_ARGS).fetchone()) # EXPLAIN QUERY PLAN reports no timing
    return plan, rows, ms

```

```

def index_experiment(conn):
    stages = [("before", *probe(conn))]
    conn.execute(INDEX)
    stages.append(("after", *probe(conn)))

```

```

print(f"\nProbe: sensor {PROBE_ARGS[0]}, {PROBE_ARGS[1][:10]}, best of {REPEATS} warm runs")
for label, plan, rows, ms in stages:
    print(f"{label:<8}{rows:>8,} rows{ms:>10.2f} ms    {plan}")

```

----- Run -----

```

def main():
    DB.unlink(missing_ok=True)          # lab.db is rebuilt from data.txt, never edited in place
    conn = connect()
    conn.executescript(SCHEMA.read_text())

    for key, value in cleaning_report(conn, load(conn)).items():
        print(f"{key:<26}{value:>14,}")
    index_experiment(conn)
    conn.close()

if __name__ == "__main__":
    main()

```

Code: Week2\HW2\src\query.py

uv run python src/query.py

```

import re
import time
from pathlib import Path

from load import connect

QUERIES = Path("sql/queries.sql")
PREVIEW = 5

def blocks(body):
    parts = re.split(r"^--\s*name:\s*(\S+)\s*$", body, flags=re.M)
    return list(zip(parts[1::2], parts[2::2]))    # ['', name, sql, name, sql, ...]

# ----- Run every named query -----

def main():
    conn = connect()
    for name, sql in blocks(QUERIES.read_text()):
        t = time.perf_counter()
        cur = conn.execute(sql.strip().rstrip(";"))
        rows = cur.fetchall()
        ms = (time.perf_counter() - t) * 1000

        print(f"\n## {name}    {len(rows):,} rows, {ms:,.0f} ms")
        print(" | ".join(d[0] for d in cur.description))
        for r in rows[:PREVIEW]:
            print(" | ".join(str(x) for x in r))
    conn.close()

if __name__ == "__main__":
    main()

```

REPORT: Week2\HW2\REPORT.md

```

# Report
Yihao Yin (yihaoyin)

## Part 1 Schema
### Results

```

```
| | rows | columns |
|---|---:|---|
| sensors | 54 | sensor_id INTEGER PRIMARY KEY, x_m REAL, y_m REAL |
| readings | 9119212 | sensor_id INTEGER (FK), ts TEXT, ts_unix REAL, epoch INTEGER, variable TEXT (CHECK),
  value REAL |
```

Discussion

sensors is loaded from mote_locs.txt but not from SELECT DISTINCT moteid, because those are different lists: only 53 of the 54 deployed motes ever transmitted. readings is long format, which makes it easy to select one channel and cheap to add another variable. One column per channel buys the opposite: the four channels of a reading sit side by side, so comparing a temperature against its own voltage is one WHERE rather than a self-join.

The cost is that every query pays for rows it does not want: query (a) needs temperature only but reads all rows to use only 1/4 of them. ts, ts_unix and epoch are each stored four times over.

SQLite has no date type, so the instant is stored twice. ts is fixed-width ISO-8601 in UTC and it sorts chronologically; ts_unix is a number because a RANGE frame needs one. Both tables are STRICT, so a value of the wrong type is rejected instead of being stored anyway.

The natural key is (sensor_id, ts, variable). I first tried (sensor_id, epoch, variable), which the dataset documentation calls monotonically increasing per mote, but it loses 448435 rows because motes restart the counter after a reboot.

PRAGMA foreign_keys = ON is issued wherever a connection is opened, because it is per-connection and off by default.

Part 2 Load and Cleaning

Results

```
| rule | rows |
|---|---:|
| in file | 2313682 |
| moteid empty, rejected | 526 |
| moteid outside the 1-54 roster, rejected | 9866 |
| unparseable timestamp or value | 0 |
| accepted | 2303290 |
| empty channel fields, recorded as absent | 93948 |
| readings written | 9119212 |
| foreign key violations | 0 |
Load time 22.2 s. Plausible bounds, applied at read time in query (d):
| channel | bound | basis | readings outside |
|---|---|---|---:|
| humidity | 0-100 %RH | dataset doc, "ranging from 0-100%" | 299084 |
| voltage | 2.0-3.0 V | dataset doc, lithium ion, "ranging from 2-3" | 2746 |
| temperature | 0-50 degC | general working environment | 408133 |
| light | 0-2000 lux | observed to saturate at 1847.36 lux | 0 |
```

Discussion

Every row of data.txt has eight fields separated by exactly one space, and a channel that did not arrive leaves its field empty, so two spaces appear in a row. Splitting on a single space keeps that empty field and every row comes out with eight. Splitting on any amount of whitespace, which is what line.split() and pandas do by default, reads those two spaces as one separator, so 93879 rows lose a field and every value after the gap shifts one column left with no error raised. An empty field means the measurement is missing rather than zero, so the loader writes no row for it.

Readings outside these bounds are kept in the database, neither deleted nor marked. The bounds are written directly into query (d) and applied when it runs, so changing one gives a different answer without reloading anything.

1847.36 lux is the largest value in the file and repeats 139078 times, far more often than any other, so the sensors reach their maximum there and keep reporting the same number. Everything below that can be real, since some sensors sit near windows, so the bound is set at 2000 rather than lower.

The dying-battery anomalies are counted but not cleaned out. Temperatures outside 0-50 degC come with a voltage averaging 2.19 V in the same transmission, against 2.55 V for the rest. A voltage rule cannot be derived from the temperatures, since the ones that would set it are the corrupted readings themselves; and temperatures cannot be judged by the voltage, since the documentation says the voltage reading itself changes with temperature and over 200000 readings below 2.40 V carry normal temperatures. Dropping everything below some voltage would throw away good data to remove bad.

Part 3 Indexing

Method

The probe query counts the readings from sensor 22 on 2004-03-09. It is run once to warm the cache and then five more times, and the fastest of those five is reported. EXPLAIN QUERY PLAN reports no timings, so the timing comes from time.perf_counter() in Python. The index is created after the load rather than in schema.sql, which is why an unindexed table exists to measure. Both configurations return the same

10452 rows, so only the cost changed.

Results

```
| index | best | EXPLAIN QUERY PLAN |
|---|---|---|
| none | 397.97 ms | SCAN readings |
| (sensor_id, ts, variable) | 0.19 ms | SEARCH readings USING COVERING INDEX ux_readings (sensor_id=? AND ts=? AND ts<?) |
```

Discussion

SCAN readings means SQLite reads every row in the table and keeps the ones that match, so a question about one mote on one day costs a walk over all 9119212 rows. SEARCH means it uses the index instead, a second copy of (sensor_id, ts, variable) kept in sorted order, so it can jump straight to that sensor and then to that day inside it. It reads about 10452 entries rather than the whole table, which is the difference between the two timings.

The part in parentheses says which conditions the index actually handled, and here it handled all three. That depends on column order: sensor_id comes first, so one mote's readings sit together and its day is a contiguous run inside them. An index built as (ts, sensor_id) would constrain only ts and read that day for all 54 motes, thirty-seven times as many rows. COVERING means every column the query needs is already in the index, so the table is never opened.

Part 4 The Three-Way Comparison

Method

All three engines answer the same question, the average temperature of each of the 53 motes that reported one. Only the query itself is timed, warm, best of 5. Lines of code counts the expression that produces the answer rather than the setup, so the pandas read_parquet call is not included.

Results

```
| engine | before it can answer | query | lines of code |
|---|---|---|---|
| pandas, Parquet into RAM | 0.5 s read | 105.5 ms | 1 |
| SQLite, lab.db | 22.2 s load | 1019.9 ms | 2 |
| DuckDB, Parquet | none | 19.0 ms | 2 |
```

Discussion

pandas and SQLite return exactly the same numbers, and DuckDB differs from them by at most $1.33e-12$. That is a different order of addition rather than a different answer: pandas and SQLite accumulate row by row, DuckDB adds partial sums from vectorised chunks, and floating-point addition is not associative. DuckDB is also the fastest and needs nothing prepared, since it reads the Parquet tree in place, while SQLite is slowest because the long schema makes it walk the whole table.

Part 5 When You Would Choose Each Store

A row store keeps one reading's fields next to each other on disk, and it reads in fixed-size pages, so a page holds whole rows. Averaging the value column means loading pages that carry every other column too, because value was never stored on its own. A column store writes all the values of one column together, so reading value reads only value. The Parquet tree is 95 MB against lab.db's 1038 MB.

SQLite is good at touching a small part of the data. Reading one sensor over one day takes a fraction of a millisecond because the index leads straight to those rows, and correcting a single reading is one UPDATE where the same fix in Parquet means rewriting the whole day's file. The file is also the data, so a value written a second ago is already in the next query. And it is the only one of the two that checks anything: the foreign key, STRICT and the CHECK reject a bad mote, a wrong type or an unknown channel at insert time.

I would keep the data in SQLite while it is still arriving or being corrected, and whenever a question names one mote or one day. I would go to Parquet and DuckDB once the data is finished and the questions change shape, scanning months but naming two or three columns. The deciding factor is the access pattern rather than the size of the data, and since the export takes 0.9 s there is no reason to pick only one.

AI Use

Generative AI was used to fix bugs in the code, to resolve SQL syntax problems, and to improve the wording and organisation of this report; every number here was produced by the scripts in this repository on my own machine.

Summary line

The script hash should match the checksum published beside the script.

```
{"andrew_id": "yihaoyin", "generated": "2026-09-24 22:06 \u00e1\u00e8\u00f0\u00e4\u0065f6", "auto_score": 4.5, "auto_of": 5.0, "report_points": 1.0, "held_for_ta": 0.0, "assignment_total": 6.0, "groups": {"table": 1.0, "baseline": 1.0, "models": 1.25, "honest": 1.25, "numbers": 0.0}, "script_sha256": "76151d2e3fb565930aed34c3b78d3a4f8debede7fcdcae3d053cfd814d921ab8"}
```